

Operating Systems

A Complete Course Presentation

Units 1 - 5



SAGE
UNIVERSITY
—BHOAL—

UNIT 3

Concurrency Control & Deadlock

Mutual Exclusion, Semaphores, IPC, Classical Synchronization Problems, Deadlock



SAGE
UNIVERSITY
—BHOPL—

Principles of Concurrency

Concurrency arises when multiple processes/threads execute at the same time, interleaved on one CPU or truly parallel on multiple CPUs

Concurrent processes may be independent or cooperating — cooperating processes share data and can affect or be affected by each other

Key difficulties: sharing of global resources, managing resource allocation optimally, locating programming errors that only appear with certain interleavings

A Race Condition occurs when several processes access and manipulate shared data concurrently, and the outcome depends on the particular order of execution

A Critical Section is the part of a program where shared data/resources are accessed and must not be executed by more than one process at a time



Requirements for Mutual Exclusion

Mutual Exclusion — only one process may be in its critical section at any time

Progress — a process outside its critical section must not block other processes from entering theirs

Bounded Waiting — a limit must exist on how many times other processes enter their critical section before a waiting process gets its turn

No assumption on relative process speeds or number of CPUs should be required for the solution to work

A process must not remain in its critical section indefinitely



Mutual Exclusion: Software Approaches

Early Software Solutions

Strict Alternation — processes take turns using a shared turn variable; simple but forces unnecessary waiting

Peterson's Algorithm — combines a turn variable with per-process interest flags to guarantee mutual exclusion for two processes

Satisfies mutual exclusion, progress and bounded waiting for two processes using only shared memory

Limitations

Relies on busy waiting (spinning), which wastes CPU cycles

Difficult to extend correctly to more than two processes

Depends on the memory model providing atomic reads/writes to shared variables

Compilers/CPU may reorder instructions, breaking correctness on real hardware without extra care



Mutual Exclusion: Hardware Support

Interrupt Disabling — a process disables all interrupts before entering its critical section and re-enables them after; works only on uniprocessors and is unsafe for user processes

Test-and-Set Instruction — an atomic hardware instruction that reads and modifies a memory location in one indivisible step, allowing a simple busy-wait lock

Compare-and-Swap (CAS) — atomically compares a memory value to an expected value and swaps it if they match; the basis for many lock-free algorithms

Hardware instructions provide the atomicity needed to build higher-level synchronization tools such as locks and semaphores



Semaphores

A semaphore is an integer variable accessed only through two atomic operations: wait() (P, decrement) and signal() (V, increment)

Counting Semaphore — value can range over an unrestricted domain; used to control access to a resource with multiple instances

Binary Semaphore (Mutex Lock) — value is restricted to 0 or 1; used for mutual exclusion around a critical section

A process calling wait() on a semaphore with value 0 is blocked and placed on a waiting queue until another process calls signal()

Semaphores avoid busy waiting by suspending blocked processes instead of looping



Pipes and Message Passing

Pipes — a unidirectional (or half-duplex) communication channel that connects the output of one process to the input of another, typically between related processes

Named Pipes (FIFOs) — extend the pipe concept so unrelated processes can also communicate, via a named entity in the file system

Message Passing — processes communicate by exchanging messages using send() and receive() primitives, without needing shared memory

Message passing can be direct (naming the sender/receiver) or indirect (via shared mailboxes/ports), and blocking or non-blocking

Well suited to distributed systems, where shared memory is not available across machines



Signals and Monitors

Signals — a software mechanism used by an OS to notify a process that a specific event has occurred (e.g. SIGINT, SIGTERM, SIGKILL); the process can catch, ignore, or use a default handler

Monitors — a high-level synchronization construct: a module encapsulating shared data, and the procedures that operate on that data, with built-in mutual exclusion

Only one process can be active inside a monitor at any time — the compiler/runtime enforces this automatically

Condition Variables inside a monitor allow processes to wait() for a condition to become true and signal() others waiting on it

Monitors are generally easier and less error-prone to use correctly than raw semaphores



Classical Problem: Producer-Consumer

Also called the Bounded-Buffer Problem: a producer generates data items and places them into a shared, fixed-size buffer; a consumer removes items from the buffer

The producer must not add to a full buffer; the consumer must not remove from an empty buffer

Solved using two counting semaphores — empty (counts free slots) and full (counts filled slots) — plus a mutex for exclusive buffer access

A classic illustration of coordinating processes that share a fixed-capacity resource



Classical Problem: Readers-Writers

Multiple reader processes may read shared data concurrently without conflict, since reading does not modify data

Writer processes require exclusive access — no other reader or writer may access the data while a writer is writing

First Readers-Writers Problem: no reader should wait unless a writer already has access (may starve writers)

Second Readers-Writers Problem: once a writer is ready, it writes as soon as possible (may starve readers)

Typically implemented with semaphores and a shared reader-count variable protected by a mutex



Classical Problem: Dining Philosophers

Five philosophers sit at a round table, alternating between thinking and eating; a single fork is placed between each pair of adjacent philosophers

A philosopher needs both the left and right fork to eat, and must put both down before thinking again

Illustrates deadlock and starvation risk: if every philosopher picks up their left fork at the same time, all wait forever for their right fork (deadlock)

Solutions: allow at most four philosophers to sit simultaneously, require picking up forks in a fixed global order, or use an arbitrator/waiter process

A widely-used benchmark example for illustrating resource allocation and deadlock in concurrent programming



Principles of Deadlock

A deadlock is a state where a set of processes are each waiting for an event (typically resource release) that only another waiting process in the set can cause

Mutual Exclusion — at least one resource must be held in a non-shareable mode

Hold and Wait — a process holding at least one resource is waiting to acquire additional resources held by others

No Preemption — resources cannot be forcibly removed from a process; they must be released voluntarily

Circular Wait — a closed chain of processes exists, each waiting for a resource held by the next process in the chain

All four conditions must hold simultaneously for a deadlock to occur



Deadlock Handling: Prevention vs Avoidance

Deadlock Prevention

Ensures at least one of the four necessary conditions can never hold

Eliminate Hold and Wait — require a process to request all resources at once, before execution begins

Allow Preemption — forcibly take resources away from a waiting process

Impose a total ordering on resource types and require requests in increasing order (breaks circular wait)

Deadlock Avoidance

Requires advance knowledge of the maximum resources each process may request

The system dynamically checks resource-allocation state before granting a request, only granting requests that keep the system in a safe state

Banker's Algorithm — a classic avoidance algorithm that simulates allocation to verify safety before committing it

More flexible than prevention, but needs more runtime information and computation



Deadlock Detection and Recovery

Detection — the system allows deadlocks to occur, then periodically checks for them, typically using a resource-allocation graph or wait-for graph

A cycle in the resource-allocation graph indicates a deadlock (with single-instance resource types)

For multiple-instance resource types, an algorithm similar to the Banker's Algorithm is used to detect deadlock

Recovery by Process Termination — abort all deadlocked processes, or abort them one at a time until the deadlock cycle is broken

Recovery by Resource Preemption — preempt resources from some processes and give them to others, rolling the preempted process back to a safe state



System Calls: signal, kill

signal() — allows a process to specify how it should handle a particular signal: perform the default action, ignore it, or run a custom handler function

kill() — despite its name, sends any specified signal to a process or group of processes (not only termination signals)

Common signals: SIGINT (interrupt from keyboard, e.g. Ctrl+C), SIGTERM (polite termination request), SIGKILL (forced, un-catchable termination), SIGCHLD (child process status changed)

Signals provide a lightweight, asynchronous mechanism for the OS or one process to notify another of events



Unit 3 Summary

Concurrency introduces race conditions; critical sections must satisfy mutual exclusion, progress and bounded waiting

Solutions range from software algorithms (Peterson's) to hardware instructions (test-and-set, compare-and-swap) to higher-level tools (semaphores, monitors)

IPC mechanisms include pipes, message passing, and signals; monitors bundle data with automatic mutual exclusion

Classical synchronization problems — Producer-Consumer, Readers-Writers, Dining Philosophers — model common coordination challenges

Deadlock requires four simultaneous conditions and can be handled through prevention, avoidance, or detection and recovery

System calls like signal and kill let processes send and handle asynchronous notifications

